

Istio From Zero

A ground-up guide to what a service mesh is for, what Istio actually does to your cluster's traffic, how it sits next to Kubernetes NetworkPolicy, and a multi-namespace lab you can run on the VKS Istio add-on to watch traffic pass and fail. Written for someone who knows Kubernetes well and Istio not at all.

- | | |
|---|---|
| 1. Why a mesh at all | 7. The three security policies |
| 2. What Kubernetes gives you without one | 8. Istio next to NetworkPolicy |
| 3. Everything Istio can do | 9. Traffic management and resilience |
| 4. Architecture: control plane and data plane | 10. North-south: internet to pod and back out |
| 5. The two modes: sidecar and ambient | 11. Observability |
| 6. Identity: how a pod proves who it is | 12. The lab: three namespaces, every gate |
| | 13. Glossary and learning order |

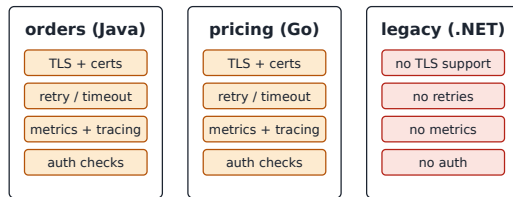
1. Why would anyone put this in a cluster?

Start with what a cluster looks like once it has a few dozen services on it, owned by several teams. Every one of those services has the same list of non-functional requirements, and none of them are the application's actual job:

- Encrypt traffic between services, and prove to the other side who you are.
- Only let the services that should call you, call you.
- Retry a failed call, give up after a timeout, stop hammering a service that is already down.
- Emit request metrics and traces so someone can see what is slow.
- Shift 5% of traffic to a new version before shifting 100%.

There are two ways to get all of that. Build it into every application, in every language, and keep it consistent across teams. Or put a proxy in the path of every connection and have the platform do it once.

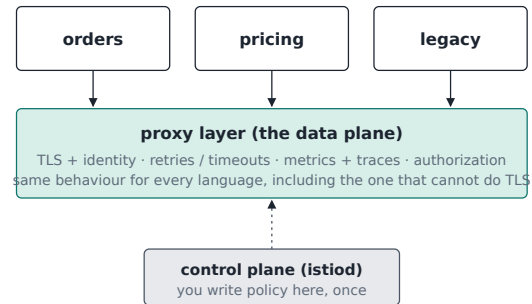
WITHOUT A MESH: EVERY TEAM BUILDS IT



Three implementations, three levels of quality, and one service that cannot do it at all.

Rotating a CA means a release of every app.

WITH A MESH: THE PLATFORM BUILDS IT ONCE



The mesh moves cross-cutting network behaviour out of application code and into a proxy layer that the platform owns. The application talks plain HTTP to localhost and everything else is done for it.

The scenarios where it pays for itself

SCENARIO	WHAT THE CUSTOMER IS ACTUALLY ASKING FOR	WHAT ISTIO GIVES THEM
Regulated workloads (bank, telecom, government)	"All traffic inside the cluster must be encrypted and we must prove it to the auditor."	Automatic mTLS between every pod, with certificates rotated every 24 hours and a policy (<code>PeerAuthentication STRICT</code>) that makes plaintext impossible. Auditor gets a YAML file, not a promise.
Zero trust between teams on a shared cluster	"The payments team's pods must only be callable by the checkout service, not by anything that happens to be on the network."	Authorization based on <i>identity</i> (which service account), not IP. IPs change every deploy; identities do not.
Safe releases	"Send 5% of users to v2, watch the error rate, then 50%, then 100%. Roll back in seconds if it goes wrong."	Weighted routing and per-version metrics, with no change to the app or its Service.
Fragile dependencies	"When the inventory service is slow, the whole site goes down."	Timeouts, retries with budgets, and outlier detection that stops sending to a bad replica, all configured outside the app.

SCENARIO	WHAT THE CUSTOMER IS ACTUALLY ASKING FOR	WHAT ISTIO GIVES THEM
Visibility with no code changes	"We need latency and error rates per service, and we cannot get every team to instrument their code."	Every request through the proxy is counted and timed. Same metrics for all languages.
Legacy applications	"This vendor app does not support TLS and is not going to be rewritten."	The proxy encrypts on its behalf. The app still speaks plain HTTP to localhost.
Controlled egress	"Pods must only reach the external APIs we have approved, and we want to see who called what."	External services registered by name, routed through an egress proxy that applies policy and logs the calling identity. Combined with NetworkPolicy to make it a real boundary (section 8).

When it is the wrong tool

- **A handful of services with no compliance driver.** A gateway for ingress plus NetworkPolicy is simpler and enough.
- **You only need "who can talk to whom" at the IP and port level.** That is NetworkPolicy's job and it is already there.
- **The team cannot own another control plane.** A mesh is another thing to upgrade, monitor and debug. If nobody will own it, it becomes the thing that breaks at 3am.
- **Pure batch or GPU training clusters.** Little service-to-service traffic, and the proxy overhead buys nothing.

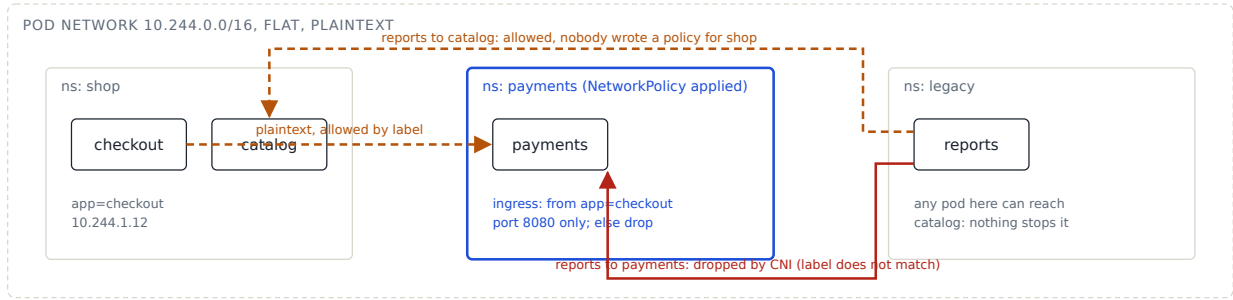
Your SOCAR-style customer with a fragmented estate, consultants coming and going, and a compliance regime is the classic case: they need identity-based policy that survives redeploys and proof of encryption. That is what they are buying, not "a service mesh".

2. What Kubernetes gives you without a mesh

You cannot understand what Istio adds until you are precise about what is already there. Three facts about a stock cluster:

1. **The pod network is flat.** Every pod can open a TCP connection to every other pod's IP, across namespaces and nodes, by default. Namespaces are a naming and RBAC boundary, not a network one.

2. **Everything is plaintext.** Unless the application itself does TLS, traffic between pods is readable by anyone on the node or the fabric.
3. **A Service is a load balancer, not a firewall.** A ClusterIP is a virtual IP that the CNI (Antrea on VKS) rewrites to a pod IP. It does not know who is calling.



----- plaintext ——— NetworkPolicy ——— dropped

NetworkPolicy is the only traffic control a stock cluster has. It is enforced by the CNI at L3/L4, it selects callers by pod label or IP block, and it is default-allow until the first policy selects a pod. It cannot see who the caller really is (a label is trivially copied), cannot encrypt, and cannot see HTTP.

Hold on to this picture. Everything in the rest of the guide is Istio adding a second layer on top of it, and section 8 is about how the two layers combine.

3. Everything Istio can do, in one table

Istio is one control plane and a set of proxies, but people use it for very different reasons. This is the full menu. You will not use all of it, and your customer asked for roughly the first three rows.

CAPABILITY	WHAT IT MEANS IN PRACTICE	RESOURCES YOU WRITE	ENFORCED BY
Workload identity	Every pod gets an X.509 certificate that says which service account it runs as. Issued and rotated automatically.	Nothing; it is automatic once a pod is in the mesh	istiod CA, ztunnel or sidecar

CAPABILITY	WHAT IT MEANS IN PRACTICE	RESOURCES YOU WRITE	ENFORCED BY
Mutual TLS	Traffic between pods is encrypted and both ends verify the other's certificate. Can be tolerated (PERMISSIVE) or required (STRICT).	<code>PeerAuthentication</code>	ztunnel (ambient) or sidecar
Authorization, L4	"Only service account X in namespace Y may connect to this pod on port Z."	<code>AuthorizationPolicy</code> (source principals, namespaces, ports)	ztunnel or sidecar
Authorization, L7	"Only GET on /api/orders, and only with a valid JWT from our issuer."	<code>AuthorizationPolicy</code> (methods, paths, headers) + <code>RequestAuthentication</code>	waypoint (ambient) or sidecar
Traffic routing	Split traffic by weight, header, or path; route to a specific version; mirror traffic to a shadow deployment.	Gateway API <code>HTTPRoute</code> (preferred), or <code>VirtualService</code> + <code>DestinationRule</code>	waypoint, gateway, or sidecar
Resilience	Per-route timeouts, retries, connection limits, ejecting unhealthy replicas (outlier detection).	<code>HTTPRoute</code> timeouts/retries, <code>DestinationRule</code> for connection pools and outlier detection	waypoint, gateway, or sidecar
Fault injection	Deliberately add latency or return errors to a percentage of requests to test how callers cope.	<code>VirtualService</code> fault section	waypoint or sidecar
Observability	Request count, latency, error rate, and TCP bytes per source/destination pair, with the caller's identity attached. Optional distributed tracing and access logs.	<code>Telemetry</code> (to tune), otherwise automatic	every proxy exports Prometheus metrics

CAPABILITY	WHAT IT MEANS IN PRACTICE	RESOURCES YOU WRITE	ENFORCED BY
Ingress gateway	An Envoy at the edge terminating TLS and routing into the cluster. Your customer uses Envoy Gateway for this instead, which is fine.	Gateway API <code>Gateway</code> + <code>HTTPRoute</code>	gateway pods
Egress control	Register external hosts, route them through a proxy that applies policy, originates TLS, and logs who called.	<code>ServiceEntry</code> , <code>waypoint</code> , <code>DestinationRule</code>	ztunnel DNS proxy + egress waypoint
Multi-cluster	One mesh spanning clusters, so a service in cluster A can call cluster B by name with mTLS and failover. Not asked for here; know it exists.	Multi-cluster install, east-west gateways	istiod + gateways

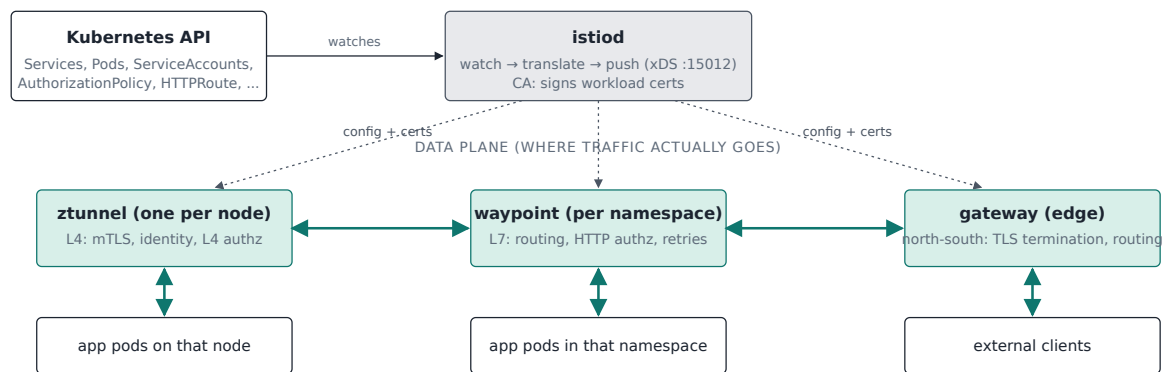
The single most important idea: **identity replaces IP**. Every capability in the security rows works on "which service account is calling" rather than "which address is calling". That is what makes policies survive redeploy, autoscaling and node failures.

4. Architecture: one brain, many hands

Istio has exactly two kinds of component. The **control plane** is a single deployment called `istiod`. The **data plane** is a set of proxies that sit in the traffic path. `istiod` never touches your traffic; it only tells the proxies what to do.

`istiod` does four jobs, and it helps to name them because errors and logs refer to them:

- **Config watcher.** Watches Kubernetes for Services, Endpoints, Pods, and every Istio resource you create.
- **xDS server.** Translates all of that into Envoy and ztunnel configuration and pushes it over gRPC (port 15012). "xDS" is the family of discovery APIs: listeners, routes, clusters, endpoints, secrets.
- **Certificate authority.** Signs a certificate for every workload, embedding its service account identity. This is the Citadel component in older docs.
- **Webhook.** In sidecar mode, mutates pods on creation to inject the proxy. In ambient mode, this job mostly disappears.



.... configuration and certificates — application traffic (mTLS)

istiod is off the data path. If istiod goes down, existing traffic keeps flowing with the last configuration the proxies received; what stops is new config pushes and, after certificates expire (24 hours by default), new connections.

What happens when you apply a policy

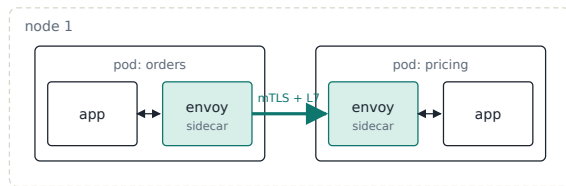
1. You `kubectl apply` an `AuthorizationPolicy` in namespace `shop`.
2. istiod's watcher sees it within a second.
3. istiod works out which proxies care: the ztunnels on nodes that run `shop` pods, and the `shop` waypoint if it has L7 rules.
4. Those proxies get an xDS push with the new rule. The proxy applies it to the next connection or request. No pod restarts.

This is why `istioctl` has so many `proxy-config` and `ztunnel-config` subcommands: they ask a proxy what it was last told, which is often different from what you think you applied.

5. The two modes: sidecar and ambient

The question "where does the proxy run?" has two answers, and it changes almost everything operationally. Istio supported only sidecars for years; ambient became the recommended mode for new installs from 1.22 onwards. The VKS add-on supports both, sidecar by default, ambient by a setting.

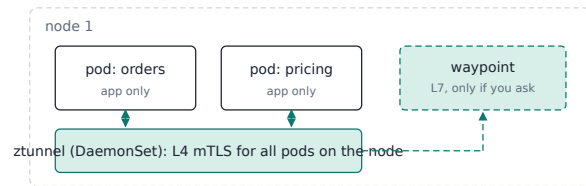
SIDECAR MODE: PROXY INSIDE EVERY POD



Consequences

- every pod carries ~50-100 MB extra memory and a CPU tax
- enrolling or upgrading means restarting every pod
- full L7 features on every workload, always
- injection needs a namespace label + pod restart
- init-container / iptables quirks with Jobs and CronJobs
- most mature; every feature and every doc assumes it

AMBIENT MODE: PROXY ON THE NODE



Consequences

- pods are untouched: no restart to enrol, no per-pod overhead
- enrol a namespace with one label, instantly
- L4 (mTLS, identity, L4 authz) everywhere for almost free
- L7 only where you deploy a waypoint, per namespace
- upgrading ztunnel touches every pod on that node at once
- newer; some features and docs still sidecar-first

Same outcome, different placement. Sidecar puts a full Envoy in every pod. Ambient splits the job: a lightweight Rust proxy (ztunnel) per node does the secure L4 transport, and a shared Envoy (waypoint) per namespace does L7 only where someone asks for it.

DECISION	SIDECAR	AMBIENT
How a workload joins	Label namespace <code>istio-injection=enabled</code> , then restart pods	Label namespace <code>istio.io/dataplane-mode=ambient</code> , done
Where mTLS terminates	Inside the pod (sidecar)	On the node (ztunnel), traffic to the pod is over loopback inside its netns
Where L7 policy runs	Every sidecar	Waypoint, if the namespace or service opts in via <code>istio.io/use-waypoint</code>
Resource cost	Per pod	Per node plus per waypoint
Upgrade blast radius	One pod at a time (rolling restarts)	Node at a time (ztunnel DaemonSet); istio-cni also node-wide
Traffic routing resources	VirtualService + DestinationRule (Gateway API also works)	Gateway API HTTPRoute preferred, attached to the waypoint
Your customer's choice	Ambient: no pod restarts in a brownfield estate, and waypoints only where L7 policy is needed. It is also why they asked for a canaried istio-cni rollout: the node-wide blast radius.	

6. Identity: how a pod proves who it is

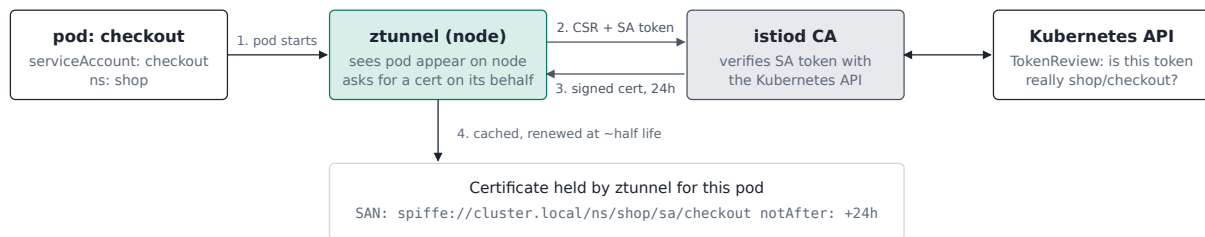
Everything in the security half of Istio rests on one mechanism, so it is worth being exact. A pod does not "have" an identity; its **Kubernetes service account** is the identity, and Istio issues a certificate that says so.

The certificate carries a SPIFFE ID, which is a URI in a fixed shape:

```
spiffe://cluster.local/ns/shop/sa/checkout
      ^trust domain   ^namespace ^service account
```

Three consequences:

- Two pods with the same service account are the same identity. If you want to distinguish workloads in policy, give them different service accounts. This is the first thing to fix in most clusters, where everything runs as `default`.
- The trust domain (`cluster.local` by default) is the root of trust. Two clusters with different trust domains do not trust each other unless you set that up.
- IP addresses appear nowhere. A policy that says "allow `checkout` " keeps working when the pod is rescheduled, scaled, or moved to another node.



The CA never trusts the proxy's word. It trusts the service account token, which the Kubernetes API vouches for. That is why the customer's `caTrustedNodeAccounts` line matters when using cert-manager's `istio-csr`: in ambient mode the requester is ztunnel, not the pod, so the CA has to be told that ztunnel's service account may request certificates for other identities.

The three CA options the customer asked about

- **istiod self-signed (default).** istiod generates its own root at install. Simplest; the root lives in a Secret in `istio-system`. Fine for a lab and for many production clusters where the mesh is its own trust island.
- **Plug-in CA.** You give istiod an intermediate certificate and key issued by your corporate PKI, stored in the `cacerts` Secret. istiod signs workload certs with it. Chains up to the enterprise root, so external systems can verify mesh identities.
- **cert-manager with istio-csr.** istiod stops signing altogether; `istio-csr` forwards CSRs to cert-manager, which uses whatever Issuer you configure (Vault, ACME, corporate CA). Most control, most moving parts.

Which of these the VKS add-on exposes as a setting is the first thing to check in its values schema. The add-on may only support the first two.

7. The three security policies

Istio has exactly three resources for security. Each answers one question. Once you can say which question you are asking, you know which one to write.

RESOURCE	QUESTION IT ANSWERS	LAYER	SCOPE OPTIONS
PeerAuthentication	Must the <i>transport</i> be mTLS? (Who is allowed to connect without a certificate?)	L4	mesh-wide (in istio-system), per namespace, per workload via selector
AuthorizationPolicy	Given who the caller is, may they do <i>this</i> ? (Connect to this port; or send this method to this path)	L4 or L7	mesh-wide, namespace, workload selector, or <code>targetRefs</code> to a waypoint Gateway or Service
RequestAuthentication	Does the <i>end user's</i> JWT come from a trusted issuer? (Not the pod's identity: the human's or the API client's)	L7	namespace or workload; needs a waypoint or sidecar

PeerAuthentication: PERMISSIVE vs STRICT

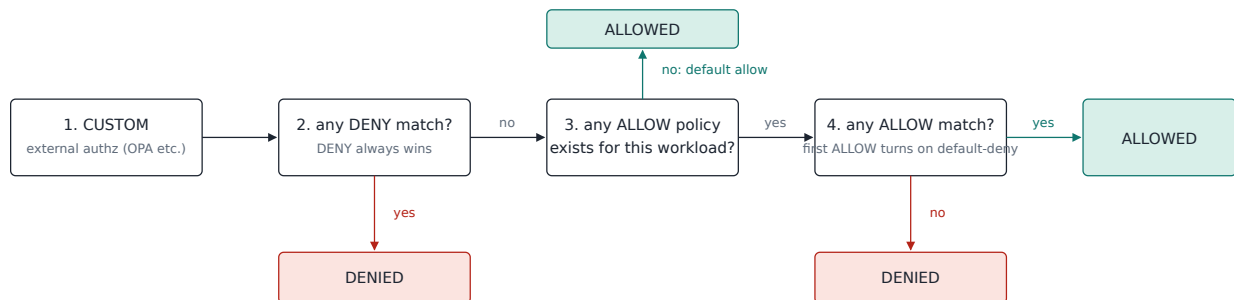
This is the "are we tolerating plaintext" switch. PERMISSIVE is the default and means the proxy accepts both mTLS and plaintext inbound. STRICT means mTLS only.

```
apiVersion: security.istio.io/v1
kind: PeerAuthentication
metadata:
  name: default
  namespace: payments      # applies to every workload in payments
spec:
  mtls:
    mode: STRICT           # PERMISSIVE | STRICT | DISABLE
```

The reason a brownfield migration keeps PERMISSIVE for a long time: a caller that is not in the mesh (unenrolled namespace, an external monitoring agent, a node-level health check) speaks plaintext. STRICT rejects it at the TCP level with a reset, before any policy is evaluated. You flip to STRICT per namespace, once you know every caller of that namespace is enrolled.

AuthorizationPolicy: the evaluation order

This is the one people get wrong, so learn the algorithm rather than the examples. For each request the proxy collects every AuthorizationPolicy that applies to the destination workload, then:



Step 3 is the trap. A workload with no ALLOW policies accepts everything. The moment one ALLOW policy selects it, everything not explicitly allowed is denied. Applying a single "allow checkout" policy to payments silently cuts off every other caller, including your monitoring.

```
apiVersion: security.istio.io/v1
kind: AuthorizationPolicy
metadata:
  name: payments-callers
  namespace: payments
spec:
  selector:
    matchLabels:
      app: payments          # which pods this protects (L4, enforced by
                             ztunnel)
  action: ALLOW
  rules:
  - from:
    - source:
        principals: ["cluster.local/ns/shop/sa/checkout"] # identity, not IP
      to:
        operation:
          ports: ["8080"]
```

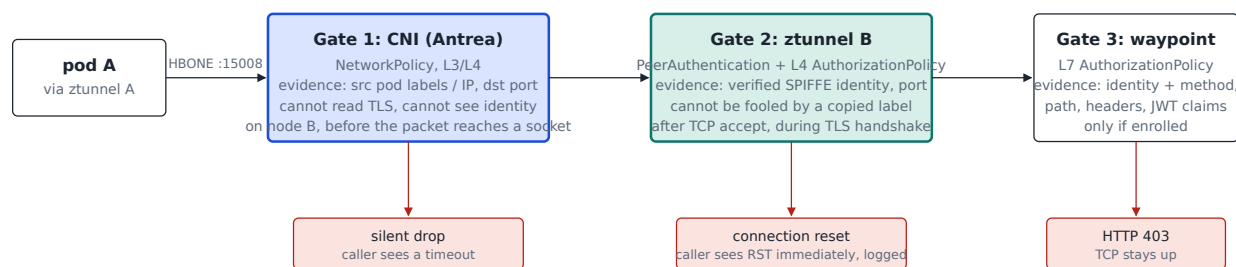
The same resource with `methods: ["GET"]` or `paths: ["/api/*"]` under `operation` becomes an L7 policy. ztunnel cannot evaluate HTTP fields. In ambient mode, an L7 rule on a workload with no waypoint is treated as unmatched and the connection is denied, which is the safe failure but a confusing one. Attach L7 policies to the waypoint with `targetRefs` instead of a pod selector.

Why plaintext callers vanish under principal rules. A caller outside the mesh arrives without a certificate, so it has no principal. A rule with `principals:` can never match it. Under PERMISSIVE the connection was accepted at step 2b of the previous page, then step 4 denies it. This is the single most common "it worked yesterday" ticket after the first AuthorizationPolicy goes in.

8. Istio next to NetworkPolicy

These are not competitors and one does not replace the other. They enforce at different places in the packet's life, on different evidence. The cleanest way to hold it:

NetworkPolicy decides whether a packet may arrive; Istio decides whether the connection inside that packet is trusted.



Order is fixed: CNI first, always. A NetworkPolicy drop never reaches Istio, so it never appears in Istio metrics or logs.

— NetworkPolicy (CNI) — Istio — refusal

Three gates, three kinds of evidence, three failure symptoms. When debugging, the symptom tells you which layer refused: timeout means CNI, reset means ztunnel, 403 means waypoint.

	NETWORKPOLICY	ISTIO AUTHORIZATIONPOLICY
Enforced by	CNI on the node (Antrea, in the kernel datapath)	ztunnel (L4) or waypoint/sidecar (L7), in user space
Identifies the caller by	Pod labels, namespace labels, IP blocks	Cryptographic identity (service account) from the mTLS cert
Can it be spoofed?	Labels: by anyone who can create a pod with that label. IPs: by anyone on the segment.	Only by stealing the service account token or the CA key
Sees HTTP?	No	Yes, with a waypoint

	NETWORKPOLICY	ISTIO AUTHORIZATIONPOLICY
Applies to non-mesh pods?	Yes, everything	Only workloads in the mesh; a policy on an unenrolled pod does nothing
Controls egress to the internet?	Yes, by CIDR. This is the real egress boundary.	Only for traffic Istio routes; ztunnel passes unknown hosts through
Default when none exists	Allow all	Allow all (until the first ALLOW policy selects the workload)
Failure symptom	Timeout (silent drop)	RST (L4) or 403 (L7), logged with the reason
Cost	Near zero	Proxy CPU, small per-connection latency

How to use both without them fighting

- **NetworkPolicy as the coarse fence.** Default-deny ingress and egress per namespace. Allow DNS to kube-system. Allow ingress from the gateway namespace. Allow egress to the internet only from the namespace that hosts the egress waypoint. Keep it label-based and boring.
- **Istio as the fine lock.** Within what NetworkPolicy allows, say which identities may call which service, and at L7, which operations.
- **Do not duplicate.** A rule that exists in both places has to be changed in both places. Pick the layer that owns each decision.

The ambient-specific gotcha with NetworkPolicy. In ambient mode, traffic between meshed pods arrives at the destination pod IP on **port 15008** (the HBONE tunnel), not on the application port. A NetworkPolicy that allows ingress only on port 8080 will drop the tunnel and every meshed call will time out. Rules for meshed workloads must allow 15008. Rules for unenrolled workloads keep using the app port. ztunnel's DNS proxy also needs your egress rules to allow UDP/TCP 53 to kube-dns as usual, since it forwards to it.

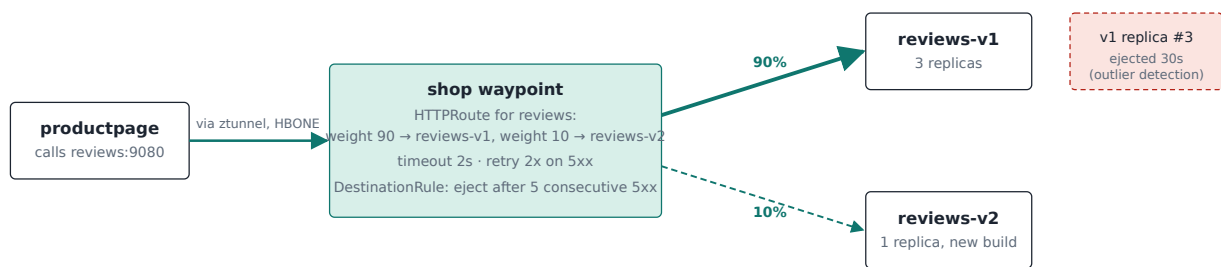
```
# NetworkPolicy for a meshed namespace: allow HBONE from the shop namespace, DNS
out, nothing else
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: payments-fence
  namespace: payments
spec:
  podSelector: {}
  policyTypes: [Ingress, Egress]
  ingress:
    - from:
      - namespaceSelector:
          matchLabels: { kubernetes.io/metadata.name: shop }
      ports:
        - port: 15008          # HBONE, not 8080, because the pods are meshed
          protocol: TCP
  egress:
    - to:
      - namespaceSelector:
          matchLabels: { kubernetes.io/metadata.name: kube-system }
      ports:
        - port: 53
          protocol: UDP
        - port: 53
          protocol: TCP
```

9. Traffic management and resilience

The security half of Istio asks "may this happen". The traffic half asks "where should it go, and what if it fails". In ambient mode all of this runs in the waypoint, so a namespace with no waypoint gets none of it.

There are two API families, and you will meet both:

- **Kubernetes Gateway API** (`Gateway` , `HTTPRoute` , `GRPCRoute` , `TCPRoute`). The newer, vendor-neutral standard. Istio's ambient docs use it throughout, and it is what your customer's Envoy Gateway also speaks.
- **Istio classic** (`VirtualService` , `DestinationRule` , `Gateway` in `networking.istio.io`). Older, more features (fault injection, mirroring, subsets), still fully supported. Most tutorials on the internet use these.



Both versions sit behind the same Kubernetes Service. Nothing in the app or the Service changed; only the waypoint's routing table did.

A canary release is the standard demonstration. Two Deployments, one Service, and a route on the waypoint that decides the split. Roll back by editing one number.

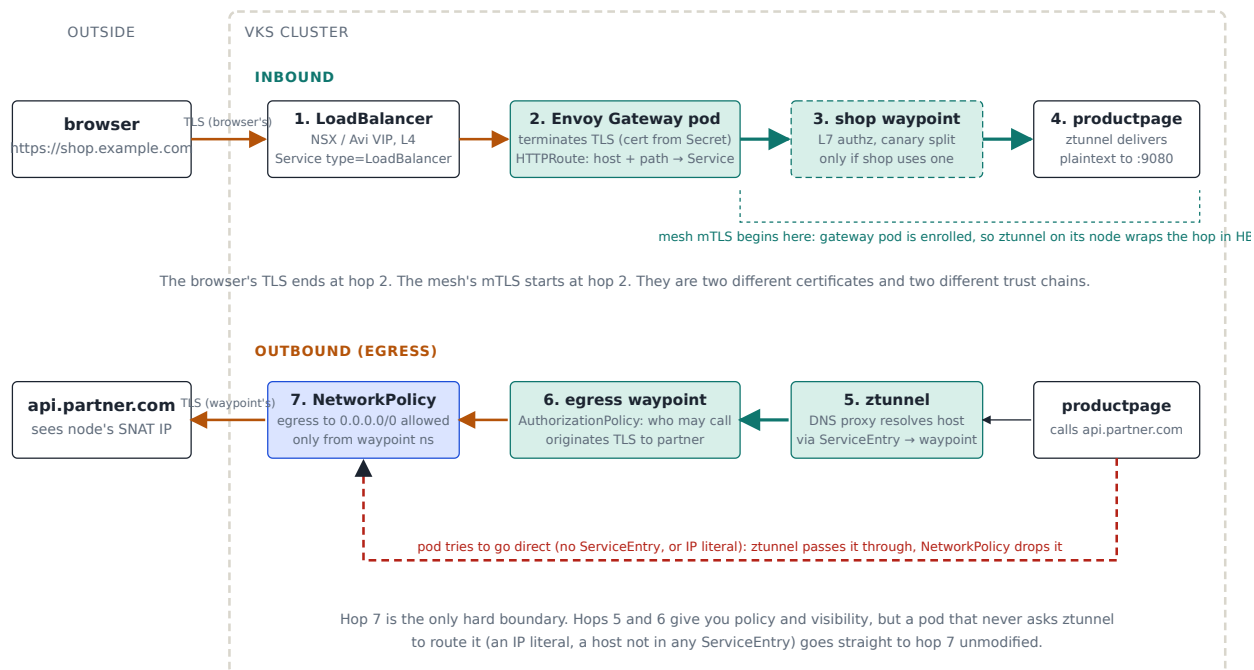
```
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: reviews-canary
  namespace: shop
spec:
  parentRefs:
  - group: ""
    kind: Service
    name: reviews          # attach to the Service; the waypoint serving it picks
    this up
    port: 9080
  rules:
  - backendRefs:
    - name: reviews-v1      # separate Services per version, or use the same
      port: 9080
      weight: 90
      Service with subsets in classic API
    - name: reviews-v2
      port: 9080
      weight: 10
    timeouts:
      request: 2s
    retry:
      attempts: 2
      codes: [500, 502, 503]
```

The resilience toolkit

MECHANISM	PROBLEM IT SOLVES	RULE OF THUMB
Timeout	A slow dependency ties up the caller's threads until the whole chain stalls	Always set one. Shorter than the caller's own timeout.
Retry	Transient errors (a pod restarting, a dropped connection) surface to the user	Only for idempotent operations; 1 to 2 attempts; with a per-try timeout. Retries multiply load on a service that is already failing.
Outlier detection	One bad replica poisons a fraction of every request	Eject after N consecutive 5xx for a short window. Keeps the rest of the pool healthy.
Connection pool limits	A burst opens thousands of connections and the backend falls over	Cap connections and pending requests per destination; excess gets a fast 503 instead of a slow collapse.
Fault injection	You want to prove the caller copes before it happens for real	Inject 500ms delay on 10% of requests in staging and watch the dashboards. VirtualService only.

10. North-south: internet to pod, and back out

East-west is pod to pod. North-south is everything that crosses the cluster edge. On VKS the pieces are: NSX or Avi providing the LoadBalancer IP, a gateway (Envoy Gateway in the customer's case, or Istio's own) terminating TLS and routing, then the mesh from there inwards. This is the whole path with what happens at each hop.



— mesh mTLS (HBONE) - - - ordinary TLS or plaintext — NetworkPolicy — blocked

Seven hops from browser to partner API. Notice that Istio never touches hops 1 and 7. The load balancer is the platform's (NSX or Avi on VKS), and the real egress boundary is the CNI's.

What each inbound hop is responsible for

- 1. LoadBalancer.** Pure L4. NSX or Avi allocates a VIP for a Service of type LoadBalancer and forwards TCP to the gateway pods. It has no idea what HTTP is. Health checks here are TCP or the gateway's readiness port.
- 2. Gateway.** Terminates the public TLS certificate (from a Secret, usually via cert-manager), reads the Host header and path, matches an `HTTPRoute`, and picks a backend Service. This is also where you would validate end-user JWTs with `RequestAuthentication` if the gateway is Istio's; with Envoy Gateway it has its own `SecurityPolicy` for that.
- 3. Waypoint.** Optional. If the destination namespace has one, the gateway's outbound call is routed through it, so the same canary and authz rules apply to external and internal callers alike.
- 4. Pod.** ztunnel on the pod's node terminates the mesh mTLS and delivers plaintext over the pod's own loopback. The app sees a normal HTTP request; the caller's identity is available in the `x-forwarded-client-cert` header if a waypoint was in the path.

Two independent TLS sessions, two CAs. The public certificate at hop 2 is from Let's Encrypt or the corporate PKI and is what the browser checks. The mesh certificate from hop 2 onwards is from istiod and is what ztunnel checks. Confusing the two is the most common misunderstanding when someone asks "is it end-to-end encrypted". The answer is yes, but as two hops, and the gateway sees plaintext in between.

Istio ingress gateway vs Envoy Gateway

Both are Envoy. Both implement the Gateway API. Istio's ingress gateway is configured by istiod and is a mesh member automatically; Envoy Gateway has its own controller and CRDs. Your customer chose Envoy Gateway and wants Istio to leave the Gateway API CRDs alone. The practical differences:

- With Istio's gateway, one control plane and one set of AuthorizationPolicy semantics from edge to pod.
- With Envoy Gateway, the edge is a separate product with its own policy CRDs (`SecurityPolicy` , `BackendTrafficPolicy`). The gateway pods can still be enrolled in ambient by labelling their namespace, which is what makes hop 2 to 4 mTLS. Confirm they have done that, or the gateway-to-pod hop is plaintext.
- Both need the Gateway API CRDs installed once. Two installers fighting over the same CRDs is the ownership problem flagged earlier.

11. Observability

Every proxy counts and times every request and connection, and labels the numbers with both ends' identities. That is the whole feature; the rest is tooling.

SIGNAL	WHERE IT COMES FROM	KEY NAMES	NEEDS APP CHANGI
L4 metrics	ztunnel :15020/stats/prometheus	<code>istio_tcp_connections_opened_total</code> , <code>istio_tcp_sent_bytes_total</code> , labelled by <code>source_principal</code> , <code>destination_service</code>	No
L7 metrics	waypoint and gateway :15020	<code>istio_requests_total</code> , <code>istio_request_duration_milliseconds</code> , labelled by <code>response_code</code> , <code>request_protocol</code>	No
Access logs	ztunnel and waypoint stdout	One line per connection or request with source, destination, identity, and the policy decision	No

SIGNAL	WHERE IT COMES FROM	KEY NAMES	NEEDS APP CHANGES
Distributed traces	waypoint and gateway, exported via OpenTelemetry	Spans per hop with <code>traceparent</code>	Yes. The app must forward trace headers from inbound or outbound request or every hop start a new trace.
Topology graph	Kiali reads the metrics above	Live service graph with mTLS lock icons and error rates	No

For your customer's SigNoz setup: SigNoz's OpenTelemetry collector scrapes the `:15020` endpoints for metrics (which is why they want the Prometheus annotations preserved) and receives traces via OTLP from the waypoints (the OpenTelemetry provider the add-on supports). Ambient L4 metrics are cheap and always on; L7 metrics only exist where a waypoint exists, which is a good argument for a waypoint even in namespaces with no L7 policy.

12. The lab: three namespaces, every gate

You have the Istio 1.28.2 add-on installed on VKS 3.6 with ambient enabled. This lab builds a small estate that has one of each kind of namespace, then walks a fixed set of test calls through every enforcement point you have read about, in the order the customer will hit them. Each stage says what to apply, what to run, what should pass, what should fail, and what the failure looks like.



Three application namespaces that differ only in their mesh status, so every test call is the same command from three places. Two infrastructure namespaces for north-south. Every pod has its own service account, because identity is the point.

Images. The manifests use `docker.io/kennethreitz/httpbin`, `curlimages/curl` and `hashicorp/http-echo`. If your lab pulls through a private registry or Harbor, rewrite the image references first. The Istio release tarball also ships these under `samples/` if you would rather use the official ones.

STAGE 0

Confirm the add-on actually gave you ambient

```
kubectl get pods -n istio-system -o wide
# expect: istiod-*, ztunnel-* (one per node), istio-cni-node-* (one per node)

kubectl get gatewayclass
# expect: istio-waypoint (and istio, if the add-on ships the ingress class)

kubectl get crd gateways.gateway.networking.k8s.io \
  -o jsonpath='{.metadata.annotations.gateway\.networking\.k8s\.io/bundle-version}'
# note the version; Istio 1.28 wants v1.2+ CRDs

istioctl version
istioctl x precheck
```

If ztunnel is not there, ambient is not enabled in the add-on values. Fix that before anything else. Also look at what the add-on let you set: this is the values schema you will compare against the customer's checklist.

```
kubectl get pkgi -A | grep -i istio          # the Carvel package install
kubectl get pkgi -n <ns> <name> -o yaml | grep -A40 values  # what you
configured
# and the schema of what is configurable:
kubectl get package -A | grep istio
kubectl get package -n <ns> istio.*1.28.2 -o jsonpath='{.spec.valuesSchema}' | jq
.
```

STAGE 1

Build the estate

```
for ns in shop partner legacy; do
  kubectl create ns $ns
  kubectl -n $ns create sa httpbin
  kubectl -n $ns create sa sleep
done

# httpbin server + sleep client in each namespace
for ns in shop partner legacy; do
cat <<EOF | kubectl -n $ns apply -f -
apiVersion: v1
kind: Service
metadata:
  name: httpbin
  labels: { app: httpbin }
spec:
  ports:
  - name: http
    port: 8000
    targetPort: 80
  selector: { app: httpbin }
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: httpbin
spec:
  replicas: 1
  selector: { matchLabels: { app: httpbin } }
  template:
    metadata:
      labels: { app: httpbin }
    spec:
      serviceAccountName: httpbin
      containers:
      - name: httpbin
        image: docker.io/kennethreitz/httpbin
        ports: [{ containerPort: 80 }]
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: sleep
spec:
  replicas: 1
  selector: { matchLabels: { app: sleep } }
  template:
    metadata:
      labels: { app: sleep }
```

```
spec:
  serviceAccountName: sleep
  containers:
  - name: sleep
    image: curlimages/curl
    command: ["/bin/sleep", "infinity"]
```

```
EOF
done
```

Now the test harness. Save this as `matrix.sh` ; you will run it after every stage.

```
#!/bin/bash
# prints a 3x3 grid: rows = caller namespace, columns = target namespace
printf "%-10s" "from\\to"; for t in shop partner legacy; do printf "%-12s" $t;
done; echo
for src in shop partner legacy; do
  printf "%-10s" $src
  for dst in shop partner legacy; do
    code=$(kubectl -n $src exec deploy/sleep -- \
      curl -s -o /dev/null -w '%{http_code}' --max-time 3 \
      http://httpbin.$dst:8000/get 2>/dev/null)
    printf "%-12s" "${code:-fail}"
  done
done
echo
done
```

Run it now, before enrolling anything. All nine cells should read `200` : a flat, plaintext cluster.

STAGE 2

Enrol two namespaces, change nothing else

```
kubectl label ns shop istio.io/dataplane-mode=ambient
kubectl label ns partner istio.io/dataplane-mode=ambient

istioctl ztunnel-config workloads
# shop and partner pods now show PROTOCOL=HBONE; legacy pods show TCP

./matrix.sh
```

FROM \ TO	SHOP	PARTNER	LEGACY	WHY
shop	200 (mTLS)	200 (mTLS)	200 (plain)	PERMISSIVE everywhere; ztunnel upgrades meshed pairs to HBONE, passes through to legacy

FROM \ TO	SHOP	PARTNER	LEGACY	WHY
partner	200 (mTLS)	200 (mTLS)	200 (plain)	same
legacy	200 (plain)	200 (plain)	200 (plain)	legacy has no ztunnel redirect; arrives plaintext on :15006, accepted under PERMISSIVE

Nothing changed for the user, and no pod restarted. Prove the encryption happened:

```
# the ztunnel on the node where shop/httpbin runs logs each inbound connection
with the peer identity
NODE=$(kubectl -n shop get pod -l app=httpbin -o
jsonpath='{.items[0].spec.nodeName}')
ZT=$(kubectl -n istio-system get pod -l app=ztunnel --field-selector
spec.nodeName=$NODE -o name)
kubectl -n istio-system logs $ZT --since=2m | grep httpbin
# look for: src.identity="spiffe://cluster.local/ns/partner/sa/sleep" on the
partner call
# and no src.identity on the legacy call

istioctl ztunnel-config certificates # every meshed SA has a cert with ~24h
validity
```

STAGE 3

STRICT on shop: the first fence

```
kubectl -n shop apply -f - <<EOF
apiVersion: security.istio.io/v1
kind: PeerAuthentication
metadata: { name: default }
spec: { mtls: { mode: STRICT } }
EOF
./matrix.sh
```

FROM \ TO	SHOP	PARTNER	LEGACY	WHY
shop	200	200	200	STRICT is about what shop <i>accepts</i> ; shop can still call out to plaintext legacy
partner	200	200	200	partner arrives at shop with a cert

FROM \ TO	SHOP	PARTNER	LEGACY	WHY
legacy	000 / reset	200	200	legacy arrives at shop in plaintext; ztunnel resets before any policy runs

Look at the failure from the caller's side to learn the symptom: `kubectl -n legacy exec deploy/sleep -- curl -v http://httpbin.shop:8000/get` gives "Connection reset by peer" almost instantly. Not a timeout. That distinction is your CNI-vs-Istio tell from section 8.

Leave STRICT in place for the rest of the lab, or remove it to keep testing the legacy path; both are valid. The customer will be in PERMISSIVE, so if you want to mirror them exactly, delete it now.

STAGE 4

L4 authorization by identity

Allow only partner's sleep to reach shop's httpbin. Note the policy selects by pod label but decides by principal.

```
kubectl -n shop apply -f - <<EOF
apiVersion: security.istio.io/v1
kind: AuthorizationPolicy
metadata: { name: httpbin-callers }
spec:
  selector: { matchLabels: { app: httpbin } }
  action: ALLOW
  rules:
  - from:
    - source:
        principals: ["cluster.local/ns/partner/sa/sleep"]
EOF
./matrix.sh
```

FROM \ TO	SHOP	WHY
shop	reset	shop's own sleep is <code>ns/shop/sa/sleep</code> , not in the list. The first ALLOW turned on default-deny for httpbin.
partner	200	the one principal allowed
legacy	reset	no cert, no principal, can never match (and STRICT would have reset it first anyway)

Now do the three experiments that teach the evaluation order:

1. Add `"cluster.local/ns/shop/sa/sleep"` to the principals list. shop→shop passes again.

2. Add a second policy with `action: DENY` matching `namespaces: ["partner"]`.
partner→shop now fails even though the ALLOW still lists it. DENY wins.
3. Delete both policies. Everything the STRICT rule permits passes again. No ALLOW policy means default allow.

Read the decision in the ztunnel log each time: `kubectl -n istio-system logs $ZT --since=1m | grep -i "policy"`.

STAGE 5

L7 without a waypoint: the fail-closed trap

```
kubectl -n shop apply -f - <<EOF
apiVersion: security.istio.io/v1
kind: AuthorizationPolicy
metadata: { name: httpbin-get-only }
spec:
  selector: { matchLabels: { app: httpbin } }
  action: ALLOW
  rules:
  - to:
    - operation: { methods: ["GET"] }
EOF
./matrix.sh
```

Every call to shop's httpbin now fails, including GETs. ztunnel cannot see the method, so an L7 rule can never match, and an ALLOW policy that never matches denies everything. This is exactly what will happen at the customer if an app team writes an L7 policy into a namespace whose waypoint has not been created yet. Delete the policy before the next stage.

STAGE 6

Add the waypoint and do L7 properly

```
istioctl waypoint apply -n shop --enroll-namespace
# this creates a Gateway named "waypoint" with gatewayClassName istio-waypoint
# and labels the namespace istio.io/use-waypoint=waypoint
```

```
kubectl -n shop get gateway,pods -l gateway.networking.k8s.io/gateway-
name=waypoint
istioctl ztunnel-config workloads | grep shop    # WAYPOINT column now filled
```

```
kubectl -n shop apply -f - <<EOF
apiVersion: security.istio.io/v1
kind: AuthorizationPolicy
metadata: { name: httpbin-get-only }
spec:
  targetRefs:
  - kind: Gateway
    group: gateway.networking.k8s.io
    name: waypoint          # attach to the waypoint, not a pod selector
  action: ALLOW
  rules:
  - to:
    - operation: { methods: ["GET"] }
EOF
```

```
kubectl -n partner exec deploy/sleep -- curl -s -o /dev/null -w '%{http_code}\n'
http://httpbin.shop:8000/get          # 200
kubectl -n partner exec deploy/sleep -- curl -s -o /dev/null -w '%{http_code}\n'
-X POST http://httpbin.shop:8000/post  # 403
```

The 403 comes with a body of `RBAC: access denied`, from Envoy in the waypoint.
Confirm with `kubectl -n shop logs deploy/waypoint | tail`. This is the customer's steady state: L4 identity policy on ztunnel, L7 method and path policy on the waypoint.

While the waypoint is up, apply the customer's defaults ConfigMap idea and watch it take effect on a fresh waypoint:

```

kubectll -n istio-system apply -f - <<EOF
apiVersion: v1
kind: ConfigMap
metadata:
  name: istio-waypoint-defaults
  labels: { gateway.istio.io/defaults-for-class: istio-waypoint }
data:
  horizontalPodAutoscaler: |
    spec:
      minReplicas: 2
      maxReplicas: 5
      metrics:
        - type: Resource
          resource: { name: cpu, target: { type: Utilization, averageUtilization:
70 } }
  podDisruptionBudget: |
    spec: { minAvailable: 1 }
  deployment: |
    spec:
      template:
        spec:
          containers:
            - name: istio-proxy
              resources:
                requests: { cpu: 300m, memory: 512Mi }
                limits: { cpu: "3", memory: 2Gi }
EOF
istiocctl waypoint delete -n shop waypoint
istiocctl waypoint apply -n shop --enroll-namespace
kubectll -n shop get hpa,pdb,deploy waypoint      # min 2 replicas, PDB present,
resources set

```

If the HPA shows `unknown` targets, metrics-server is missing. That is the dependency on their checklist.

STAGE 7

Traffic split on the waypoint

```
kubectl -n shop create sa echo
for v in v1 v2; do
cat <<EOF | kubectl -n shop apply -f -
apiVersion: apps/v1
kind: Deployment
metadata: { name: echo-$v }
spec:
  replicas: 1
  selector: { matchLabels: { app: echo, version: $v } }
  template:
    metadata: { labels: { app: echo, version: $v } }
    spec:
      serviceAccountName: echo
      containers:
      - name: echo
        image: hashicorp/http-echo
        args: ["-text=$v", "-listen=:8080"]
---
apiVersion: v1
kind: Service
metadata: { name: echo-$v }
spec:
  selector: { app: echo, version: $v }
  ports: [{ port: 8080 }]
EOF
done
# the "front door" service both versions sit behind
kubectl -n shop apply -f - <<EOF
apiVersion: v1
kind: Service
metadata: { name: echo }
spec:
  selector: { app: echo }
  ports: [{ port: 8080 }]
EOF

kubectl -n shop apply -f - <<EOF
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata: { name: echo-split }
spec:
  parentRefs:
  - { group: "", kind: Service, name: echo, port: 8080 }
  rules:
  - backendRefs:
    - { name: echo-v1, port: 8080, weight: 90 }
    - { name: echo-v2, port: 8080, weight: 10 }
EOF
```

```
kubectl -n partner exec deploy/sleep -- sh -c \
  'for i in $(seq 1 100); do curl -s http://echo.shop:8080; done | sort | uniq -
  c'
# roughly 90 v1 / 10 v2
```

Change the weights to 50/50 and re-run. Then delete the HTTPRoute: the split returns to whatever the Service's endpoints give you (50/50 by replica count). That is the difference between "load balancing" and "traffic management".

STAGE 8

NetworkPolicy meets the mesh

Apply a policy that a well-meaning platform engineer would write for shop: allow ingress on the application port from partner, nothing else.

```
kubectl -n shop apply -f - <<EOF
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: fence }
spec:
  podSelector: {}
  policyTypes: [Ingress]
  ingress:
    - from:
      - namespaceSelector: { matchLabels: { kubernetes.io/metadata.name: partner } }
    }
  ports: [{ port: 80, protocol: TCP }]
EOF
./matrix.sh
```

FROM \ TO	SHOP	SYMPTOM	WHY
partner	fail after 3s	timeout, not reset	HBONE arrives on :15008, the policy only allows :80. Antrea drops it silently before ztunnel sees it.
shop	fail after 3s	timeout	same-namespace traffic is not in the allow list at all
legacy	fail after 3s	timeout	not in the allow list

Now change `port: 80` to `port: 15008`. `partner→shop` passes again. You have just reproduced the single most common ambient-plus-NetworkPolicy outage. Also notice: while the CNI was dropping, nothing appeared in ztunnel logs or Istio metrics, because the packets never got that far.

Keep this policy in place with 15008, and add an `Egress` section allowing only DNS. That is the fence the next two stages assume.

STAGE 9

North-south: an ingress Gateway into shop

```
kubectl create ns edge
kubectl label ns edge istio.io/dataplane-mode=ambient # so gateway → pod is mTLS

kubectl -n edge apply -f - <<EOF
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata: { name: public }
spec:
  gatewayClassName: istio
  listeners:
  - name: http
    port: 80
    protocol: HTTP
    allowedRoutes: { namespaces: { from: All } }
EOF
kubectl -n edge get gateway public -w # wait for ADDRESS: the NSX/Avi VIP
kubectl -n edge get svc # public-istio, type LoadBalancer

kubectl -n shop apply -f - <<EOF
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata: { name: public-echo }
spec:
  parentRefs:
  - { name: public, namespace: edge, kind: Gateway, group:
gateway.networking.k8s.io }
  hostnames: ["echo.lab.local"]
  rules:
  - backendRefs: [{ name: echo, port: 8080 }]
EOF

VIP=$(kubectl -n edge get gateway public -o
jsonpath='{.status.addresses[0].value}')
curl -H 'Host: echo.lab.local' http://$VIP/ # from your laptop: v1 or v2
```

Now update the shop NetworkPolicy to also allow ingress from `edge` on 15008 (the gateway pod is meshed, so it also arrives over HBONE). Until you do, the curl from outside times out at the gateway with a 503 or hangs, which is the north-south version of stage 8.

Extend it: add a `tls` listener on 443 with a self-signed cert in a Secret, and observe that the browser's TLS ends at the gateway while the ztunnel log on the echo node still

shows `src.identity=.../ns/edge/sa/public-istio` . Two TLS sessions, two CAs, as in section 10.

If the add-on does not ship the `istio` GatewayClass, install Envoy Gateway instead to mirror the customer, and enrol its namespace the same way.

STAGE 10

Egress boundary

You need something outside the cluster to call. In an air-gapped lab, a VM on the same segment running `python3 -m http.server 8080` is enough; call it `ext.lab.local` in your DNS or just use its IP in the ServiceEntry.

```

# 1. prove the pod can currently leave: passes, because nothing stops it
kubectl -n shop exec deploy/sleep -- curl -s -o /dev/null -w '%{http_code}\n'
http://ext.lab.local:8080/

# 2. fence egress: shop may only talk to DNS and to the egress namespace
kubectl -n shop apply -f - <<EOF
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: egress-fence }
spec:
  podSelector: {}
  policyTypes: [Egress]
  egress:
    - to: [{ namespaceSelector: { matchLabels: { kubernetes.io/metadata.name: kube-
system } } }]
      ports: [{ port: 53, protocol: UDP }, { port: 53, protocol: TCP }]
    - to: [{ namespaceSelector: { matchLabels: { kubernetes.io/metadata.name:
egress } } }]
    - to: [{ namespaceSelector: { matchLabels: { kubernetes.io/metadata.name:
partner } } }]
    - to: [{ namespaceSelector: { matchLabels: { kubernetes.io/metadata.name: shop
} } }]
EOF
# same curl now times out: the CNI is the boundary

# 3. give it a governed way out
kubectl create ns egress
kubectl label ns egress istio.io/dataplane-mode=ambient
istioctl waypoint apply -n egress --name egress-wp

kubectl -n egress apply -f - <<EOF
apiVersion: networking.istio.io/v1
kind: ServiceEntry
metadata:
  name: ext-lab
  labels: { istio.io/use-waypoint: egress-wp }      # route this host via the
egress waypoint
spec:
  hosts: ["ext.lab.local"]
  ports: [{ number: 8080, name: http, protocol: HTTP }]
  resolution: DNS
  location: MESH_EXTERNAL
EOF

# the waypoint must be allowed to leave; egress ns has no NetworkPolicy so it can
kubectl -n shop exec deploy/sleep -- curl -s -o /dev/null -w '%{http_code}\n'
http://ext.lab.local:8080/    # 200 again

# 4. and now policy on who may call outside
kubectl -n egress apply -f - <<EOF

```

```

apiVersion: security.istio.io/v1
kind: AuthorizationPolicy
metadata: { name: ext-callers }
spec:
  targetRefs: [{ kind: ServiceEntry, group: networking.istio.io, name: ext-lab }]
  action: ALLOW
  rules:
    - from: [{ source: { principals: ["cluster.local/ns/shop/sa/sleep"] } }]
EOF
kubectl -n partner exec deploy/sleep -- curl -s -o /dev/null -w '%{http_code}\n'
http://ext.lab.local:8080/ # 403 (partner is fenced? no: it has no egress
policy, so it goes direct and passes)

```

That last line is the lesson. partner has no egress NetworkPolicy, so its pod reaches ext.lab.local directly, never touching the waypoint, and gets 200 not 403. The AuthorizationPolicy only governs traffic that was routed to the waypoint. Apply the same egress-fence to partner and it becomes 403. Istio is the policy; the CNI is the boundary; you need both.

Flagged, verify: whether `meshConfig.outboundTrafficPolicy.mode: REGISTRY_ONLY` is honoured by ztunnel in 1.28. If the add-on exposes that setting, test it here by removing the NetworkPolicy and seeing whether an unregistered host is still passed through. Do not tell the customer either way until you have seen it.

STAGE 11

See it in metrics

```

# L4, from any ztunnel
kubectl -n istio-system port-forward ds/ztunnel 15020:15020 &
curl -s localhost:15020/stats/prometheus | grep
istio_tcp_connections_opened_total | grep shop
# labels include source_principal and destination_service: identity-attributed
metrics for free

# L7, from the shop waypoint
kubectl -n shop port-forward deploy/waypoint 15020:15020 &
curl -s localhost:15020/stats/prometheus | grep istio_requests_total
# response_code="403" rows are your L7 denies from stage 6

```

If you have the Prometheus Supervisor service running, add a scrape job for pods annotated `prometheus.io/scrape: "true"` on port 15020 and path `/stats/prometheus`; that is exactly what the customer's SigNoz collector will do. Kiali (`samples/addons/kiali.yaml` in the release tarball) reads those metrics and draws the graph with mTLS padlocks; worth 20 minutes for the mental picture.

STAGE 12

Break it, then upgrade it

- **Kill a ztunnel.** `kubectl -n istio-system delete pod $ZT` . Run the matrix in a loop. Calls to pods on that node fail for the few seconds until the DaemonSet replaces it. This is the blast radius the customer is worried about.
- **Remove the namespace label.** `kubectl label ns partner istio.io/dataplane-mode-` . partner instantly becomes plaintext; under STRICT, shop stops accepting it. No pod restarted. This is why the customer insists on owning that label.
- **Change an add-on value.** Edit the PackageInstall values (something harmless, like a log level) and watch `kubectl -n istio-system get pods -w` . Note whether ztunnel and istio-cni roll one node at a time or all at once. That observation is your answer to the customer's canary request; if the add-on redeploys everything at once, you can only offer the DaemonSet's `maxUnavailable` as a mitigation, if the values expose it.
- **Cert expiry.** Scale istiod to 0, wait, and note that traffic keeps working. Certificates last 24 hours, so you will not see the failure unless you wait that long or shorten the lifetime in mesh config. Scale it back up. Understanding that istiod is not on the data path is worth this five-minute experiment.

Debugging cheat sheet

QUESTION	COMMAND
Is this pod in the mesh, and does it have a waypoint?	<code>istioctl ztunnel-config workloads</code>
What identity does it have, and when does the cert expire?	<code>istioctl ztunnel-config certificates</code>
What policies does ztunnel think apply?	<code>istioctl ztunnel-config policies</code>
Why was a connection refused at L4?	<code>kubectl -n istio-system logs <ztunnel on dest node></code>
Why was a request refused at L7?	<code>kubectl -n <ns> logs deploy/waypoint ; for more, istioctl proxy-config log deploy/waypoint --level rbac:debug</code>
What routes does the waypoint have?	<code>istioctl proxy-config routes deploy/waypoint -n <ns></code>
Is my YAML wrong before I apply it?	<code>istioctl analyze -n <ns></code>
Is istiod healthy, and did it push?	<code>istioctl proxy-status</code>
Timeout vs reset vs 403?	timeout: CNI/NetworkPolicy. reset: ztunnel (STRICT, waypoint bypass, L4 deny). 403: waypoint L7.

13. Glossary and learning order

TERM	MEANING IN ONE LINE
Envoy	The C++ proxy that does the actual work in sidecars, waypoints and gateways. Istio is largely a control plane for Envoy.
istiod	The single control plane process: config watcher, xDS server, CA, webhook.
xDS	Envoy's family of discovery APIs. istiod speaks it to every proxy.
ztunnel	Ambient's per-node L4 proxy, written in Rust. Does mTLS, identity, L4 authz, DNS capture.
waypoint	Ambient's per-namespace (or per-service) Envoy for L7. Declared as a Gateway API Gateway with class istio-waypoint.
istio-cni	DaemonSet that installs the traffic redirect rules in each pod's netns. Chains onto Antrea; does not replace it.
HBONE	"HTTP-Based Overlay Network Encapsulation": the tunnel between ztunnels. mTLS on port 15008 carrying HTTP/2 CONNECT.
SPIFFE	The identity URI standard. Istio identities are <code>spiffe://<trust domain>/ns/<ns>/sa/<sa></code> .
trust domain	Root of the identity namespace, default <code>cluster.local</code> . Two meshes with different trust domains do not trust each other by default.
PeerAuthentication	Is mTLS required (STRICT) or optional (PERMISSIVE) for inbound to a workload.
AuthorizationPolicy	Who may do what, by identity (L4) and by HTTP attributes (L7).
RequestAuthentication	Which JWT issuers are trusted for end-user tokens. L7.
ServiceEntry	Adds a host outside the cluster to Istio's registry so it can be routed and policed.
DestinationRule	Per-destination settings: load balancing, connection pools, outlier detection, TLS origination, subsets.
VirtualService	Classic routing resource. Mostly superseded by HTTPRoute for new work, still needed for fault injection and mirroring.

TERM	MEANING IN ONE LINE
Gateway API	The Kubernetes-standard routing API (Gateway, HTTPRoute, ...). Used by Istio, Envoy Gateway, and others.
Telemetry	Resource for tuning metrics, logs and tracing per scope.
Kiali	The mesh UI: service graph, config validation, mTLS status.

Order to learn it in

1. Sections 1, 2 and 4 of this page, then stages 0 to 2 of the lab. One evening. You now have a working mesh and know what it is for.
2. Sections 6, 7 and 8, then stages 3, 4, 5 and 8. Two or three evenings. This is the security model and its relationship to NetworkPolicy, and it is 80% of what the customer's checklist is about.
3. Section 5 and stage 6, plus the waypoint defaults. One evening. You can now answer every waypoint question on their list.
4. Sections 9 and 10, stages 7, 9 and 10. Two evenings. Traffic management and the full north-south path.
5. Section 11 and stages 11 and 12. One evening. Observability and operations, which is where their SigNoz, upgrade-notice and canary-rollout items live.
6. Then read the official docs' ambient section end to end. It will make sense now, and you will notice which parts the VKS add-on constrains.

Keep the companion page, *Istio Ambient Traffic Path*, open while doing stages 3 to 6; it is the zoomed-in view of the gates you are testing.